Бърза разработка на софтуер

Цели

- Да обясни как итеративният, инкрементален процес на разработка води до по-бърза изработка и предаване на по-полезен софтуер.
- Да дискутира същественото от методите на “чевръстото” (agile) програмиране
- Да обясни принципите и практиките на екстремното програмиране
- Да обясни ролята на прототипирането в софтуерния процес.

Основни теми

- чевръсти методи
- Екстремно програмиране
- Бърза разработка на приложения
- Софтуерни прототипи

Бърза (Rapid) разработка на приложения

- Бизнес средата се изменя много бързо и бизнесът трябва да отговаря на новите възможности и конкуренция.
- За тази цел е нужен софтуер, но поради горните причини често бързото получаване на софтуера е най-критичното изискване.
- Бизнесът може да приеме и по-ниско качество, ако бързо му се предаде софтуер имащ съществената функционалност.

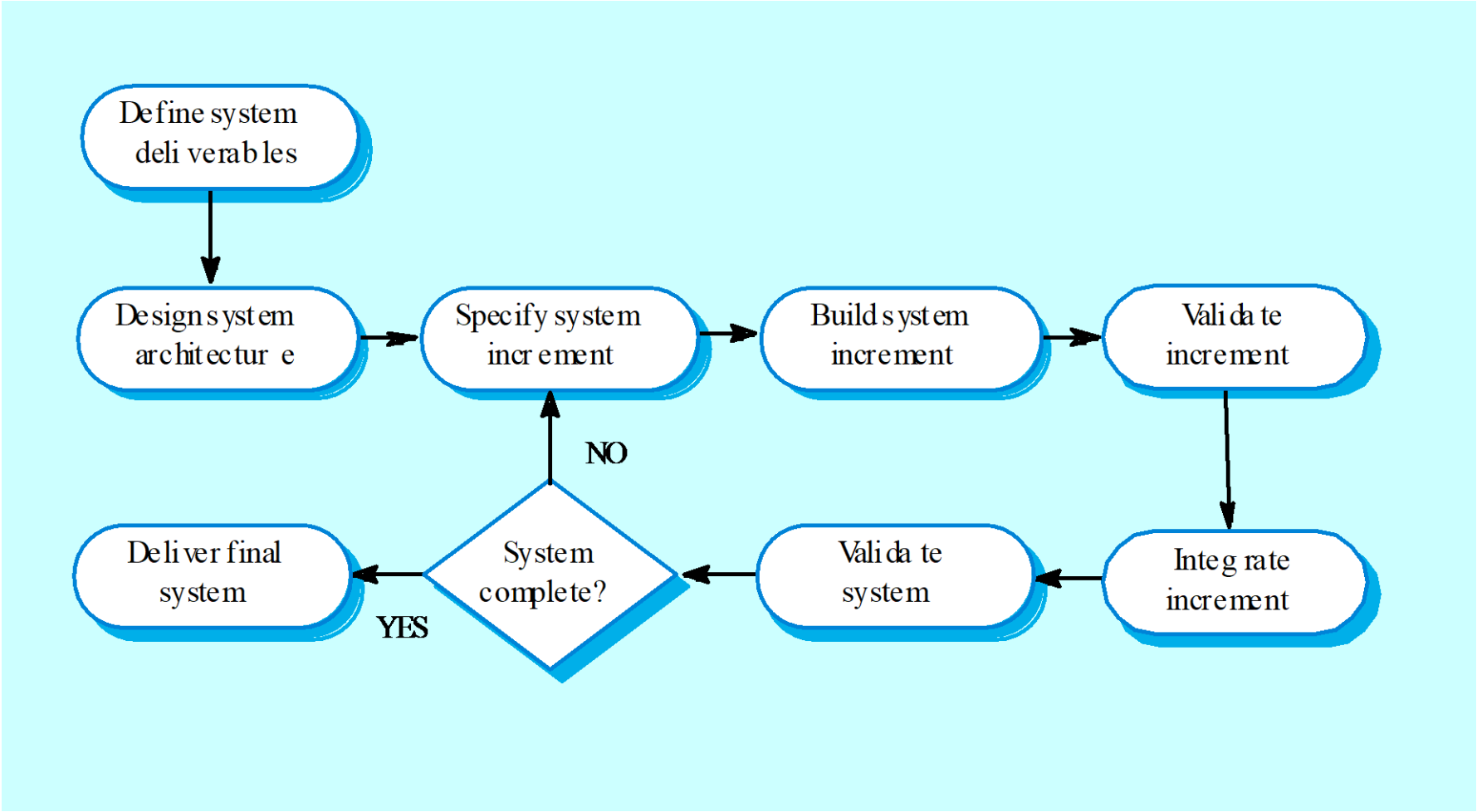
Изисквания

- Поради променливата среда, често е невъзможно да се достигне до устойчив и последователен набор от с-мни изисквания.
- Затова “водопадния” модел не е подходящ и и единственият път да се получи бързо софтуера е подходът основан на итеративна спецификация и доставка.

Характеристики на RAD процеса

- Процесите на спецификация, проектиране и реализация са паралелни. Няма детайлна спецификация и документацията е минимална.
- Системата се разработва като серия от нараствания. Крайните потребители оценяват всяко приближение и дават предложения за следващото.
- Потребителският интерфейс се разработва обикновено като се използва интерактивна с-ма за разработка.

Итеративен процес на разработка



Предимства на инкрементaлната разработка

- Ускорено предаване на функциите за клиента. Всяко приближение най-приоритетната функционалност на клиента.
- Съпричастност на потребителя. Потребителите трябва да се включат в разработката, което означава, че е по-вероятно с-мата да отговори на техните изисквания и потребителите са по-ангажирани със с-мата.

Проблеми на инкрементaлната разработка

- Проблеми на управлението

  Трудно се оценява напредъка и трудно се откриват проблемите, защото няма документация, която да покаже, какво е направено.

- Проблеми на договарянето

  Нормалният договор включва спецификация; без спецификация трябва да се използват различни форми на договори.

- Проблеми на валидирането

  Как ще се тества с-мата без спецификация?

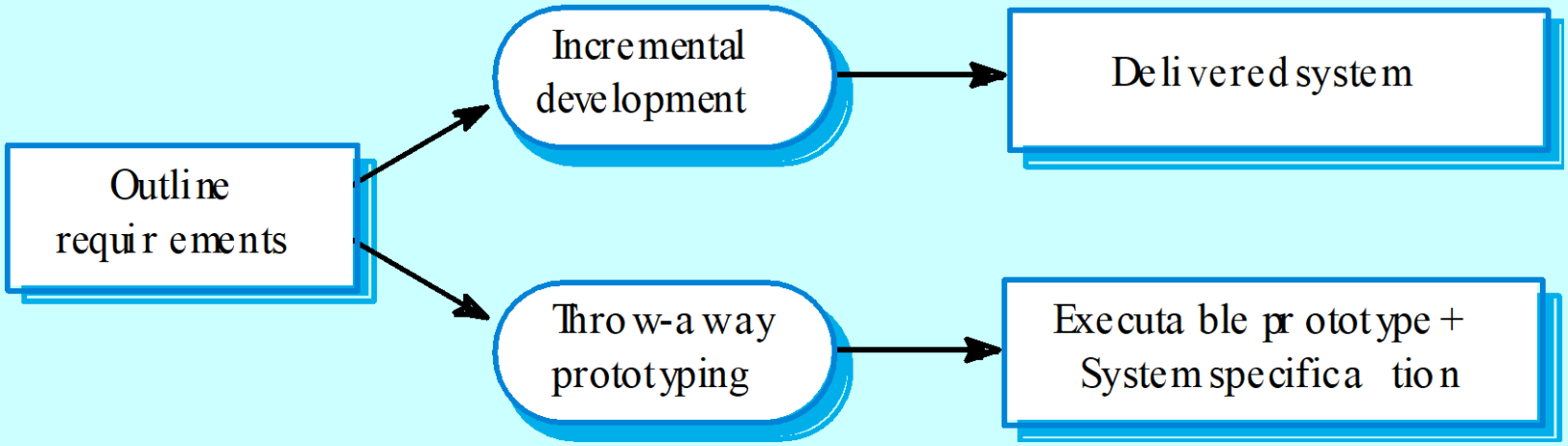
- Проблеми на поддръжката

  Непрекъснатите промени могат да развалят структурата на с-мата, което прави последната по-скъпа за промяна и развитие, за да отговори на нови изисквания

Прототипиране

- При някои големи с-ми, особено когато много екипи работят на различни места, итеративната разработка може да е неподходяща
- Прототипирането, където една експериментална с-ма се използва за база за формулиране на изискванията, може да се използва. Последната се изхвърля, когато се договори с-мната спецификация.

Инкрементална разработка и прототипиране



Несъвместими цели

- Целта на инкрементaлната разработка е да се достави работеща с-ма на крайните потребители. Разработката започва с най-ясните в момента изисквания.
- Целта на прототипирането е да валидира или извлече изискванията към с-мата. Процесът започва от най-неясните изисквания.

Гъвкави - Agile методи

- Неудовлетворението от допълнителните разходи на методите за проектиране води до създаването на “чевръстите” методи. Тези методи:
  - Се концентрират в/у кода, а не в/у проекта;
  - Основават се на итеративния подход.
  - Имат за цел да произведат софтуера бързо и да го развиват също така бързо, за да удовлетвори променящите се изисквания.
- “чевръстите” методи са може би най-подходящи за малки или средни бизнес с-ми или PC продукти

Принципи на “гъвкавите” методи

Принциоп	Обяснение
Ангажиране на потребителя	Клиентът трябва да ангажиран по време на процеса на разработка. Неговата роля е да дава и определя важността на изискванията и да оценява приближенията на с-мата.
Доставка на части	Софтуерът се разработва на приближения, като клиентът определя кои изисквания да се включат във всяко приближение.
Хората, не процеса	Трябва да се разберат и използват уменията на разработващия екип. Той трябва да се остави да развие собствени методи на работа без предписващи процеси.
Подготовка за промени	Очаква се, изискванията да се променят и с-мата се проектира така, че да може да се нагоди към тези промени
Поддържане на простота	Съсредоточава се в/у простотата на разработвания софтуер и на използвания процес. Активно се работи за отстраняване на сложността от с-мата там, където е възможно.

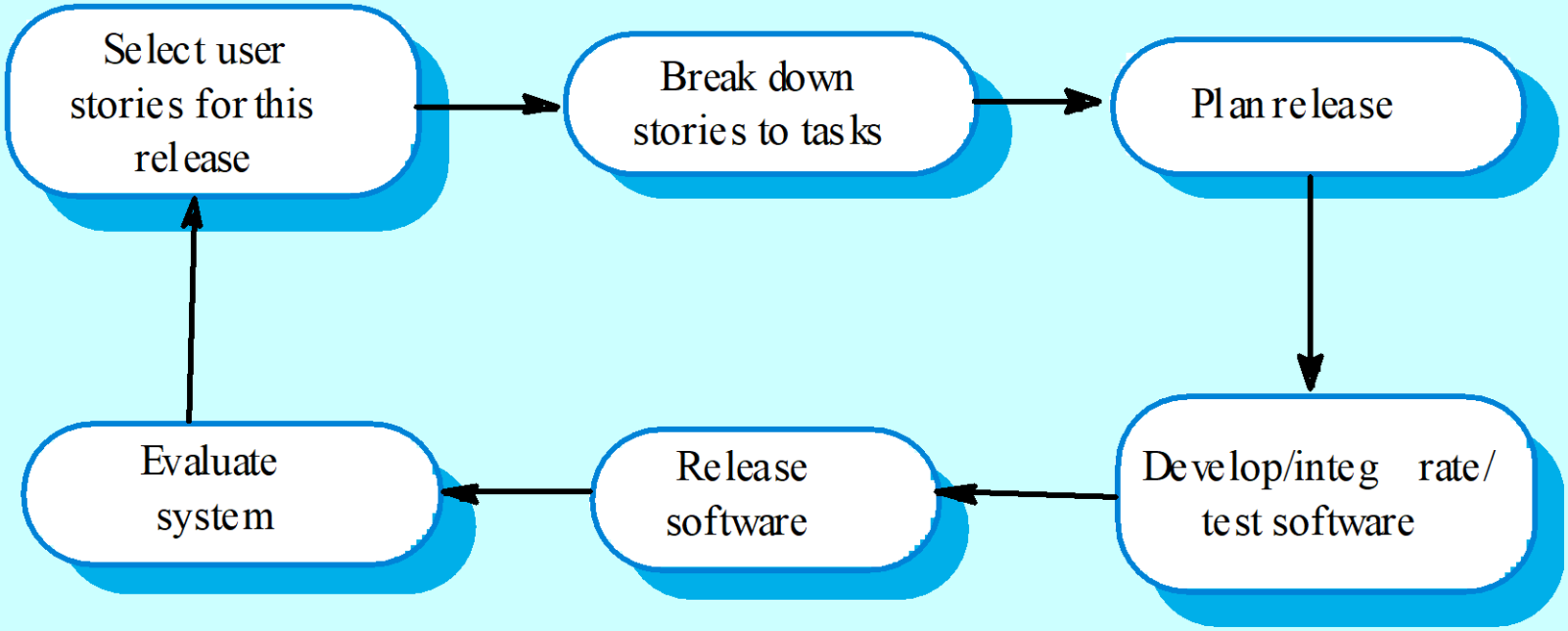
Проблеми на “гъвкавите” методи

- Може да е трудно да се поддържа интереса на клиентите, въвлечени в процеса.
- Членовете на екипа може да са неподходящи за интензивната работа, която характеризира метода
- Определянето на важността на промените може да се окаже трудно, когато има много поръчители.
- Поддържането на простота изисква допълнителна работа.
- Договарянето може да бъде проблем, както с другите подходи към итеративната разработка.

Екстремно програмиране (XP)

- Може би най известния и най-използвания “чевръст” метод
- Екстремното програмиране(XP) предлага екстремен подход към итеративното разработване.
  - Нови версии се изграждат по няколко пъти на ден;
  - Приближеният се получават от клиента всеки 2 седмици
  - Всички тестове се провеждат за всеки билд и билдът се приема, само ако тестовете са минали успешно.

Цикъл на предаване при XP



XP практики 1

Инкрементално планиране	Изискванията се записват в специални "Story Cards” и тези, които трябва да се включат в следващата част се определят от разполагаемото време и от относителния приоритет. Разработчиците разбиват изискванията на задачи “Tasks“.
Малки добавки	Първо се разработва минималното множество от функции, които са важни за бизнеса. Нови версии се разработват много често непрекъснато добавят нова функционалност към първата версия.
Прост проект	Извършва се само проектиране, достатъчно да удовлетвори текущите изисквания и не повече от това.
Тестване веднага след разработката	Използва се автоматична с-ма за тестване за написването на тестовете за новата функционалност преди да се добави към съществуващата.
Реорганизиране	Очаква се всички разработчици да оптимизират кода си непрекъснато веднага, когато се намерят възможности за подобрение. Това държи кода прост и лесен за поддръжка.

XP практики 2

Програмиране по двойки	Разработчиците работят по двойки, като си проверяват взаимно работата и си помагат.
Колективна собственост	Двойките работят по всички области на с-мата, така че няма острови на експертна разработка и всички разработчици притежават целия код. Всеки може да променя всичко.
Непрекъсната интеграция	В момента, който задачата е готова, тя се интегрира в цялата с-ма. След такова интегриране всички тестове трябва да минат успешно.
Поддържащо темпо	Многото извънредна работа не се толерира, защото чистият резултат е влошаване на качеството и намаление на средната продуктивност.
Присъствие на клиента	През цялото време представител на Клиента трябва да е на разположение на XP екипа. В процеса на разработка, клиентът е член на екипа и отговаря за предаването на изискванията на екипа за реализация.

Принципи на XP и гъвкавото програмиране

- Инкрементaлната разработка се поддържа чрез малки, чести версии.
- Въвличането на клиента означава непрекъснато присъствие на клиента при екипа.
- Хората, а не процеса, се поддържат чрез програмирането по двойки, колективната собственост и процеса, който избягва многото извънредно време.
- Промените се поддържат чрез редовни версии
- Поддържане на простотата чрез постоянно подобрение и съкращение на кода

Сценарии на изискванията

- При XP потребителските изисквания се изразяват като сценарии и потребителски разкази
- Те се пишат на картички и екипът ги разбива на реализационни задачи. Тези задачи са основа на оценките за разписанието и бюджета.
- Клиентът избира сценариите за включване в следващата версия на базата на приоритетите им и на оценките на разписанието.

Карта за сценарий за зареждане на документ

Зареждане и отпечатване на документ
Първо избирате документа за зареждане от изобразения списък.После трябва да кажете на с-мата как ще платите – това може да бъде чрез абонамент, корпоративна сметка или кредитна карта. След това получавате формуляр за авторските права, който трябва да попълните и след като го изпратите обратно, документът се зарежда във вашия компютър. След това избирате принтер и документът се отпечатва. Вие съобщавате на с-мата дали отпечатването е успешно. Ако документът е разрешен само за печат, вие не можете да запазите pdf версията на документа и тя се изтрива от вашия компютър.

XP и промените

- Здравият смисъл в софтуерното инженерство казва, че трябва да се проектира така, че да се променя. Струва си да се отдели време и усилия за очакваните промени, тъй като това намалява разходите по-късно в жизнения цикъл.
- XP поддържа, че не си струва, тъй като определени промени не могат да се очакват със сигурност.
- По-скоро то предлага непрекъсната оптимизация на кода, което да направи промените по-лесни, в случай че се наложи да се реализират.

Тестване при XP

- Предварителна разработка на теста.
- Инкрементална разработка на тестовете от сценариите.
- Въвличане на потребителя в разработката на теста и валидирането.
- Автоматични батчове за тестове се използват, за да се пуснат всички тестове при изработката на нова версия.

Карти за задачи за зареждане на документ

Задача 1. Реализация на основния алгоритъм

Задача 2. Каталог и избирането на статия

Задача 3. Реализация на плащането

Плащането може да се направи по 3 различни начина. Потребителят избира начина, по който ще плати. Ако е библиотечен абонамент, той трябва да въведе абонатния ключ, който с-мата ще провери. Освен това може да се въведе номер на сметка на организация. Ако последният е валиден, на тази сметка ще се изпрати платежно искане. Може да се въведе и 16 цифров номер на кредитна карта и срок на валидност. Те трябва да се проверят за валидност и ако са валидни, искане се изпраща до сметката на тази карта.

Описание на тест случай

Тест4: Проверка на валидността на кредитна карта
Вход: Стринг представляващ номера на картата и 2 цели числа представляващи месеца и годината на срока й на валидност. **Проверка:** Проверява дали всички байтове в стринга са цифри Проверява дали месецът е м/у 1 и 12 и дали годината е >= от текущата година. Като използва 1-те 4 цифри на картата, проверява дали издателят на картата е валиден чрез търсене в таблицата на издателите на кредитни карти. Проверява валидността на кредитната карта чрез изпращане на данните й до издателя. **Изход:** ОК или съобщение за грешка показващо, че картата е невалидна.

Предварителна разработка на теста.

- Написването на тестовете преди кода изяснява изискванията, които трябва да се реализират.
- Тестовете се пишат по-скоро като програми отколкото като данни, така че да могат да се изпълняват автоматично. Тестът включва проверка за това, че е изпълнен правилно.
- Когато се добавя нова функционалност се изпълняват всички предишни и нови тестове. По този начин се проверява дали новата функционалност не вкарала грешки.

Програмиране по двойки

- При XPпрограмистите работят по двойки на едно място и разработват код
- Това спомага за съвместната собственост на кода и разпространява знание в екипа.
- Това е неформално преглеждане, защото всеки ред код се гледа от повече от един човек.
- Той улеснява оптимизацията, от която целият екип печели.
- Измерванията показват, че производителността е подобна на тази, когато двама човека работят независимо.

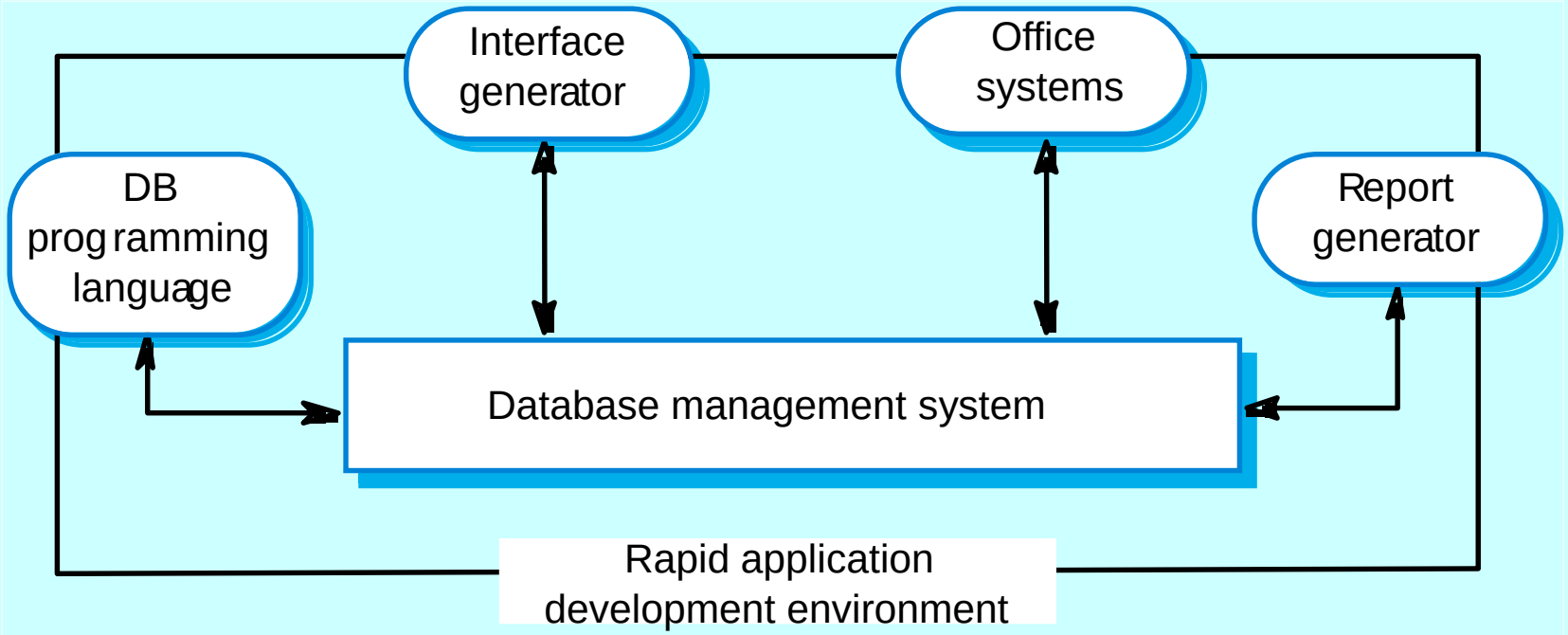
Бързо разработване на приложения (RAD)

- Гъвкавите методи се радват на голямо внимание, но отдавна се използват и други подходи за бързо разработване на приложения.
- Те са предназначени за разработване на ориентирани към данни бизнес приложения и представяне на информация от бази от данни.

Средства на RAD средите

- Програмен език за бази от данни
- Генератор на интерфейси
- Връзки до офис приложения
- Генератори на репорти

RAD среда


Interface generator
Office systems
DB programming language
Report generator
Database management system
Rapid application development environment

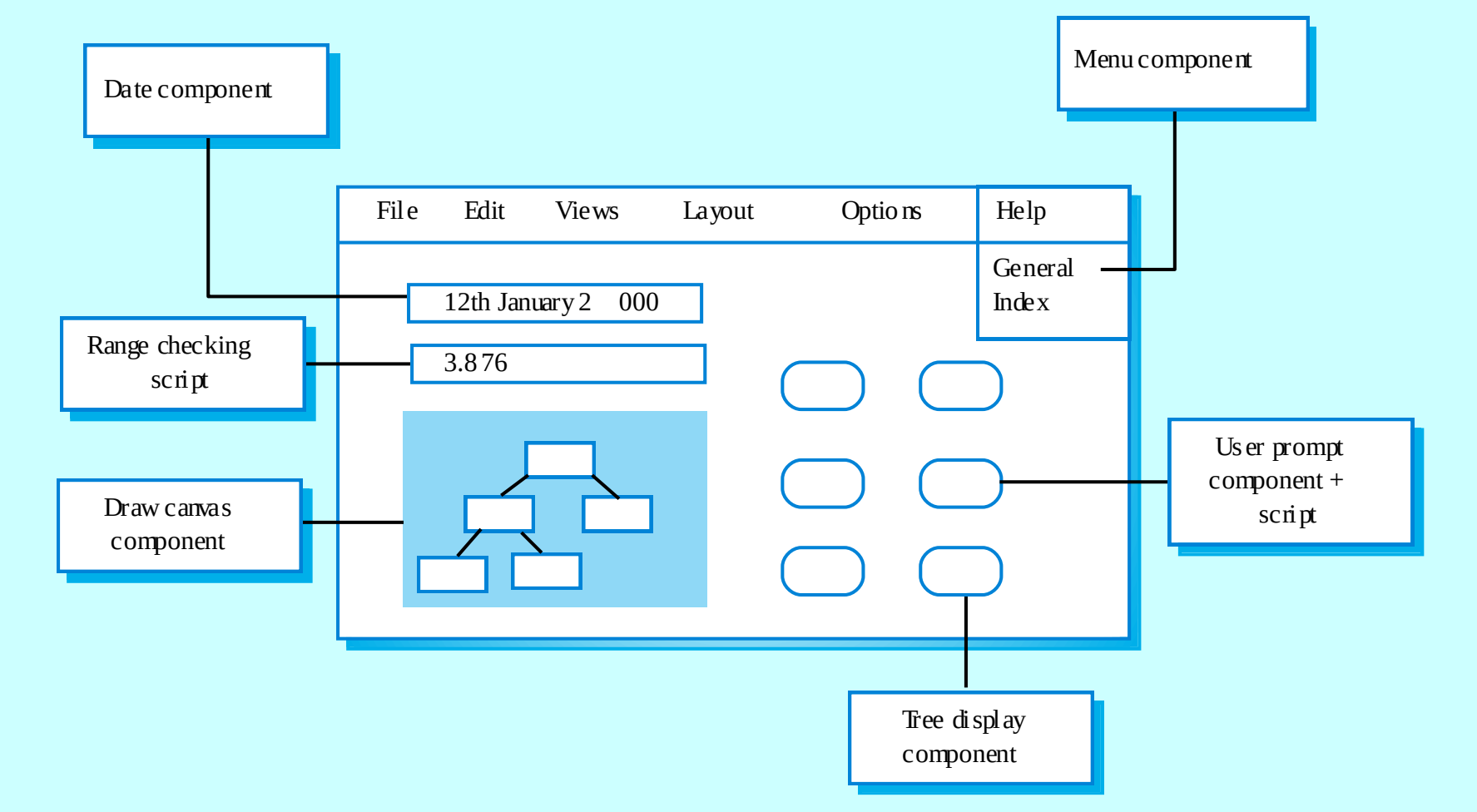
Генериране на интерфейс

- Много приложения се развиват около сложни формуляри и ръчното програмиране на тези формуляри е много бавно.
- RAD средите включват поддръжка за визуална генерация, която има:
  - Интерактивно създаване на формуляра, като се използва drag and drop техника.
  - Свързване на формулярите, ако е специфицирано последователност от формуляри
  - Верификация, ако са дефинирани разрешени интервали за стойностите на полета от формуляра.

Визуално програмиране

- Визуалното програмиране се поддържа от скриптови езици като Visual Basic (и не само скриптови), където прототипът се разработва, чрез създаване на потребителски интерфейс от стандартни елементи и асоциране на компонентите с тези елементи.
- Съществуват големи библиотеки от компоненти за този тип разработки
- Те трябва да бъдат приспособени за конкретните изисквания на приложението.

Визуално програмиране с многократно използване



Проблеми на визуалното програмиране

- Трудно се координира разработка в екип
- Няма явна архитектура на с-мата
- Сложните зависимости м/у части на програмата могат да създадат проблеми на поддръжката.

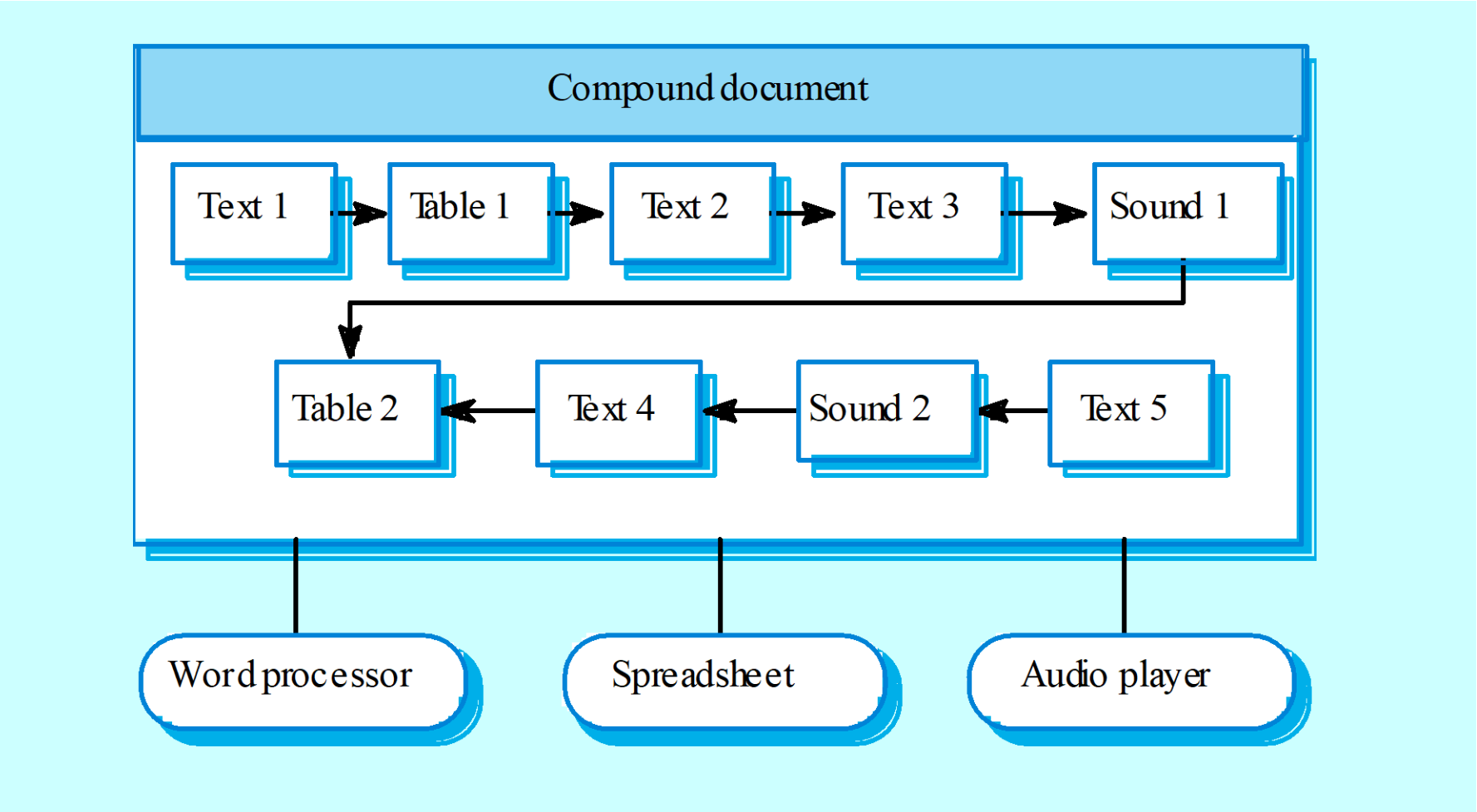
Използване на готови програмни продукти (COTS)

- Конфигурирането и свързването на готови с-ми (Commercial off-The Shelf) е ефективен подход за ефективна разработка.
- Например, с-ма за управление на изискванията може да се изгради като се използва:
  - База от данни (съхранява изискванията)
  - Текстов процесор (въвеждане на изискванията и форматиране на репорти)
  - Електронна таблица (проследяване на управлението)

Сложен (съставен) документ

- За някои приложения може да се създаде прототип чрез разработка на съставен документ.
- Това е документ с активни елементи (като електронна таблица), които позволяват потребителски пресмятания.
- На всеки активен елемент е асоциирано приложение, което се изпълнява при избор на елемента.
- Самият документ интегрира различните приложения

Свързване на приложения



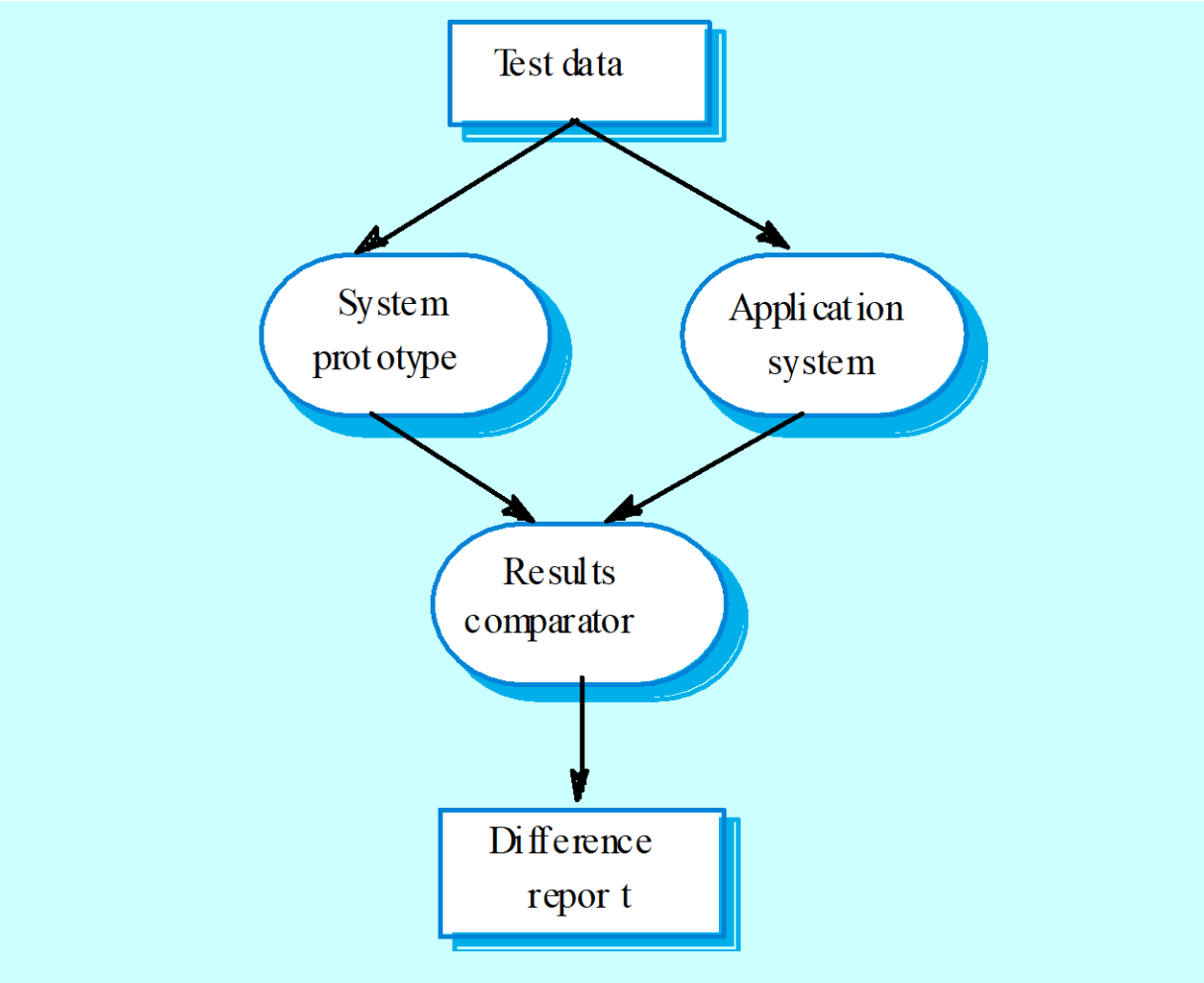
Прототипиране

- Прототипът е начална версия на с-мата, която се използва да демонстрира общата представа и да изпробва различни проектни решения.
- Той може да се използва в:
  - Процеса на инженеринг на изискванията – за извличане и валидиране на изискванията;
  - Процеса на проектиране – да изследва възможности и за разработка на потребителски интерфейс;
  - Процеса на тестването – за сравнителни (back-to-back) тестове

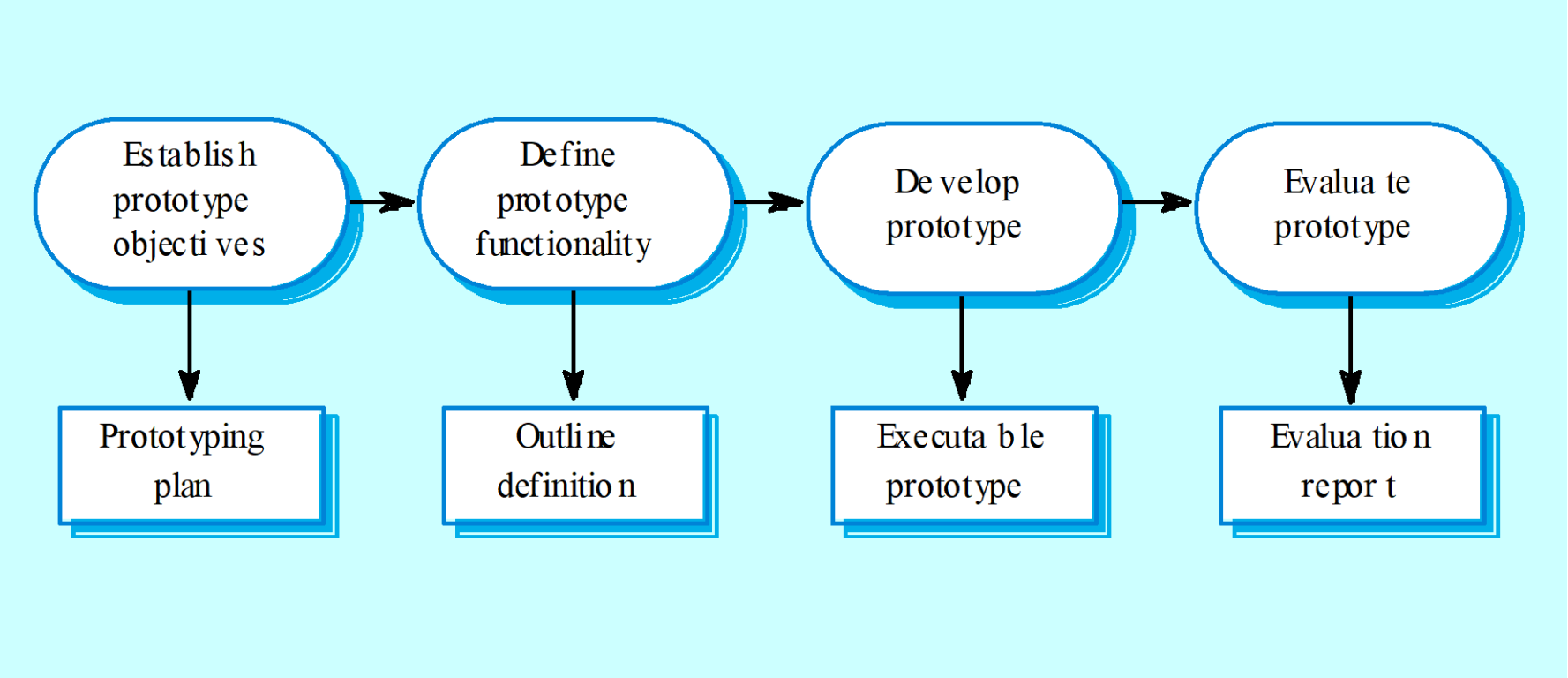
Предимства на прототипирането

- Подобрена използваемост на с-мата
- По добро съответствие с истинските нужди на потребителя
- Подобрено качество на проекта
- Подобрена възможност за поддръжка
- Намалени усилия за разработка

Сравнително (Back to back) тестване



Процес на прототипиране



Изхвърляне на прототипите

Прототипите трябва да се премахнат след процеса на разработка, тъй като те не са добра основа за реалната с-ма (особено за САМ)

- Може да е невъзможно да се настрои с-мата да отговори на нефункционалните изисквания.
- Те обикновено не са документирани
- Структурата им по принцип е влошена от непрекъснатите и бързи промени.
- Възможно е прототипът да не отговаря на стандартите за качество на организацията.

Заключение

- Итеративния подход към разработката води до по-бързо получаване на готовия софтуер.
- Чевръстите методи са итеративни методи, чиято цел е да намалят режийните разходи и допълнителното време и да направят разработката по-бърза.
- Екстремното програмиране включва практики като систематично тестване, непрекъснато усъвършенстване и въвличане на клиентите.
- Подходът за тестване при XP, при който изпълнимите тестове се разработват преди кода, е силно качество.

Заключение

- Средите за бързо разработване на приложения включват езици за бази от данни, средства за генериране на формуляри и връзки към офис приложения.
- Протипирането с изхвърляне служи да се изследват изискванията и проектните решения.
- Когато се реализира прототипиране се почва с най-малко ясните изисквания; при инкрементaлната разработка се почва с най-ясните изисквания